

Explain these

```
header {
  background-color: #FFF8F3;
  background: url("../images/developer.png"),
url("../images/header_bg.png"), #FFF8F3;
  background-position: 23px 38px, bottom right;
  background-repeat: no-repeat;
  padding-top: 50px;
}
```

Here’s what each line is doing (in simple words), and how the “multiple backgrounds” part works.

CSS Copy code

```
header {
  background-color: #FFF8F3;
  background: url("../images/developer.png"), url("../images/header_bg.png");
  background-position: 23px 38px, bottom right;
  background-repeat: no-repeat;
  padding-top: 50px;
}
```

1) background-color: #FFF8F3;

Sets a **plain background color** for the `<header>`.

- `#FFF8F3` is a very light “warm white / peach” color.
- This line is a “fallback” (if images don’t load, you still get color).

2) background: url(...), url(...), #FFF8F3;

This is the **shorthand background property**.

Here you’re setting **3 background layers** at once:

1. `url("../images/developer.png")` → **top layer** (drawn in front)
2. `url("../images/header_bg.png")` → **second layer** (behind the first image)
3. `#FFF8F3` → **bottom layer color** (behind both images)

✔ Important: In CSS, when you write multiple backgrounds separated by commas, the **first one is on top**.

Also note:

- Because you used `background: ...`, it can override parts of `background-color`. But you included `#FFF8F3` at the end, so the color is still applied.

3) `background-position: 23px 38px, bottom right;`

This sets the position of each image (in the same order as the images).

- `23px 38px` → applies to `developer.png`
Means: **23px from the left, 38px from the top**
- `bottom right` → applies to `header_bg.png`
Means: place it at the **bottom-right corner**

So it's like:

- developer image: near the top-left but slightly inside
- header background image: stuck to the bottom-right

4) `background-repeat: no-repeat;`

Stops the background images from repeating (tiling).

- Without this, a small image might repeat like a pattern across the header.
- Here: each image shows **only once**.

⚠ With multiple backgrounds, you can also write:

CSS

Copy code

```
background-repeat: no-repeat, no-repeat;
```

But writing one value (`no-repeat`) applies to all layers.

5) `padding-top: 50px;`

Adds **space inside the header** at the top.

So your header content (text, buttons, etc.) will start **50px lower**, not stuck to the top edge.

Mini example you can copy-paste (with easier paths)

CSS

Copy code

```
header{
  background:
    url("developer.png"),
    url("header_bg.png"),
    #FFF8F3;

  background-position:
    23px 38px,
    bottom right;

  background-repeat: no-repeat;
  padding-top: 50px;
}
```

If you want, paste your HTML header part too, and I'll explain how these backgrounds will *look* with your actual layout.



what is means by fallback in >> This line is a "fallback" (if images don't load, you still get color). ??

Fallback means a **backup option**.

So in your CSS:

CSSCopy code

```
background-color: #FFF8F3;
```

This color is there **just in case** the background images don't show (for example: wrong file path, slow internet, image deleted, user blocks images, etc.). Then the header won't look empty—at least it will still have that light color.

Simple example

CSSCopy code

```
.box{
  background-color: lightgray;      /* fallback */
  background-image: url("photo.jpg"); /* main */
}
```

- If `photo.jpg` loads  → you see the photo (and the color is mostly hidden behind it).
- If `photo.jpg` fails  → you still see `lightgray` .

In your code

You actually used the fallback color twice:

CSSCopy code

```
background-color: #FFF8F3;          /* fallback */
background: url(...), url(...), #FFF8F3; /* also includes the color */
```

Even if both images don't load, the header still has `#FFF8F3` .

If you want, I can show you a tiny HTML+CSS demo you can run to see the fallback working.



is this property valid??

background-position: -30px -80px;

Yes  it's valid.

background-position: -30px -80px; means:

- **X (horizontal)** = -30px → move the background image **30px to the left**
- **Y (vertical)** = -80px → move the background image **80px up**

So you're basically **shifting the background image up-left**, which can be useful if you want to show a specific part of the image inside the element.

Quick demo

html Copy code

```
<div class="box"></div>

<style>
.box{
  width: 300px;
  height: 180px;
  border: 2px solid;
  background-image: url("https://picsum.photos/600/400");
  background-repeat: no-repeat;

  /* valid */
  background-position: -30px -80px;
}
</style>
```

When negative values matter

- If your background image is **bigger than the element**, negative values let you "slide" the image so a different part becomes visible.
- If the image is **smaller**, moving it negative might push it out of view.

If you tell me your element size + image size, I can explain exactly what part will show.



